Functional and Non-Functional Requirements

Functional Requirements

A. Greenhouse/Garden Area

Plant Care

FR1: The system must manage different plant species profiles (Flower, Succulent, Tree) with specific care requirements

- **Design Pattern**:
    - Bridge Pattern

FR2: The system must select appropriate care strategies (watering, pruning, fertilizing) based on plant species

- **Design Pattern**:
    - Strategy Pattern
    - Bridge Pattern

FR3: The system must provide interchangeable care algorithms (Standard, Drip, Flood watering; Standard, Minimal, Artistic pruning)

- **Design Pattern**:
    - Strategy Pattern

FR4: The system must execute care operations through unified strategy interface

- **Design Pattern**:
    - Strategy Pattern

FR5: The system must determine care amounts and intervals from plant species profiles

- **Design Pattern**:
    - Bridge Pattern

Plant Life Cycle

FR6: The system must manage plant progression through states (Planted, InNursery, Growing, ReadyForSale, Withering)

- **Design Pattern**:
    - State Pattern

FR7: The system must automatically advance plant states based on time and care received

- **Design Pattern**:
    - State Pattern

FR8: The system must execute different behaviours in different plant states (watering in Planted, fertilizing in Growing, etc.).

- **Design Pattern**:
    - State Pattern

FR9 :The system must notify the StaffManager for when a plant requires care.

- **Design Pattern**:
    - Observer Pattern

Inventory

FR10: The system must notify the InventoryManager when plants reach maturity and are ready to be moved to the sales floor.

- **Design Pattern**:
    - Observer Pattern

Stock

FR11: The system must provide configurable construction of a complex, hierarchy plant structure(Plant object)with the ability to handle multiple plant structures uniformly .

- **Design Pattern**:
    - Builder Pattern
    - Composite Pattern

B. The Staff

Plant Care

FR12: The system must delegate staff tasks through a chain of handlers, passing requests to the next handler if busy or unable to process.

- **Design Pattern**:
    - Chain of Responsibility Pattern

FRI3: The system must encapsulate plant care actions into command objects (Water, Prune, Fertilize, MoveToSalesFloor) that can be executed by staff.

- **Design Pattern**:
    - Command Pattern

Plant Life Cycle

FR14: The system must route commands to appropriate staff members based on their role (Greenhouse staff for care commands, Sales staff for sales floor commands).

- **Design Patterns**:
    - Command Pattern
    - Chain of Responsibility

Plant Life Cycles & Inventory

FRI5: The system must manage plant lifecycle state transitions (InNursery. → Planted → Growing → ReadyForSale → Withering) with state-specific behaviours.

- **Design Pattern**:
    - State Pattern

FR16: The system must notify observers (InventoryManager, StaffManager, LifeCycleMonitor) when plants change states.

- **Design Pattern**:
    - Observer Pattern

Customer Interaction

FR17: The system must track plant inventory, staff assignments, and active tasks through a centralized singleton manager.

- **Design Pattern**:
    - Singleton Pattern (Inventory Manager)

C. The Customers and The Sales Floor

Customer Browsing

FR18: The system must allow customers to view available plants and pots from the sales floor inventory

- **Design Pattern:**
    - Singleton Pattern (InventoryManager)
    - Facade Pattern

Personalization

FR19: The system must allow customers to create base pot types that a customer would be able to customise to their liking.

- **Design Pattern:**
    - Factory Method

FR20: The system must allow customers to dynamically customise plant pots with colours, patterns, textures, and finishes.

- **Design Pattern**:
    - Decorator Pattern

FR21: The system must enable customers to build orders by selecting plants, pots, and bundles with automatic discount calculation

- **Design Pattern**:
    - Builder Pattern
    - Director Pattern

Interaction with Staff

FR22: The system must suggest customers with bouquets for different events.

- **Design Pattern**:
    - Template Method Pattern

FR23: The system must validate customer orders through staff approval and process payments through multiple payment methods

- **Design Pattern**:
    - Chain of Responsibility Pattern
    - Adapter Pattern

FR24: The system must notify staff observers of customer interactions and order events in real-time

- **Design Pattern**:
    - **Observer Pattern**

Sales and Transactions

FR25: The system must maintain customer order history with the ability to save and restore previous orders (undo orders).

- **Design Pattern**:
    - Memento Pattern

Non-Functional Requirements

NFR1: Extensibility

- The system must allow adding new care strategies without modifying existing plant or state code.

NFR2: Extensibility

- The system must support adding new plant species profiles without changing PlantProduct abstraction.

NFR3: Scalability

- The system design must scale to accommodate growing numbers of plants, staff members, and care strategies.

NFR4: Loose Coupling

- The system must decouple care algorithms from plant objects enabling runtime strategy switching.

NFR5: Flexibility

- The system must select care strategies at runtime based on plant type and current state.

NFR6: Extensibility

- The system must allow adding new lifecycle states without affecting existing state logic.

NFR7: Modularity & Loose Coupling

- The system must maintain clear separation between PlantProduct abstraction and PlantSpeciesProfile implementation.

NFR8: Extensibility

- The system must allow adding new order handlers (validation, payment, notification) without modifying existing Customer or Order code.

NFR9: Loose Coupling

- The system must decouple order construction logic from the Customer class, enabling different order types to be built without changing Customer implementation

NFR10: Maintainability

- The system must separate UI concerns from business logic through facade pattern, reducing main program complexity and improving code maintainability.